

L A B

Software Quality Engineering

Lab Environment Setup



Week 1

Install Tools · Configure Environments · Git Setup · First Test Run

CLO-1 | Psychomotor Domain | BT Level: Set (P-2)

1 Credit Hour · 3 Hours Lab / Week · 16-Week Course

Lab Session Agenda

3-Hour Session Breakdown

00:00 – 00:20	Introduction & Safety Briefing Lab rules, submission policy, platform walk-through
00:20 – 01:00	Tool Installation VS Code, Git, Python / Java JDK, pytest / JUnit setup
01:00 – 01:45	Version Control Setup Git init, GitHub account, first commit, .gitignore
01:45 – 02:30	Testing Framework Configuration Install pytest/JUnit, write first test, run and interpret
02:30 – 02:50	Verification Checklist Tool verification worksheet — confirm all tools working
02:50 – 03:00	Submission & Wrap-up Submit verification checklist, Q&A, preview Week 2

Lab Objectives & CLO Alignment

What you will achieve by the end of this session

CLO-1 (P-2 — Set)

Set up their lab environments and tools to work on SQE-related tasks. Configure testing environments, inspection processes, and quality metrics to perform software quality evaluations.

Session Learning Objectives

Install & verify:	All required tools installed: VS Code, Git, Python/Java, pytest/JUnit, coverage tool.
------------------------------	---

Configure Git:	Initialise a Git repository, create a GitHub account, make a first commit.
-----------------------	--

Run a test:	Write a trivial function, create a test file, execute it with pytest or JUnit.
--------------------	--

Document setup:	Complete and submit the tool verification checklist.
------------------------	--

Why Environment Setup Matters

The Foundation of Every Lab Activity



Reproducibility

Every student works in the same environment — same tools, same versions. This eliminates 'works on my machine' problems.



Automation-Ready

A properly configured environment enables test automation, CI/CD pipelines, and continuous quality assurance from day one.



Version Control

Git tracks every change you make. All lab submissions are version-controlled — a professional industry standard.



Early Defect Detection

Having test frameworks installed means you can write and run tests immediately — catching bugs as early as possible.

Required Tools Overview

What we are installing today

Tool	Category	Purpose	Version
VS Code	IDE / Editor	Primary code editor with extensions for Python/Java, Git, and test runners.	<i>Latest stable</i>
Git	Version Control	Track changes, collaborate, and submit lab work via GitHub.	≥ 2.40
Python 3.x	Language Runtime	Required for pytest-based lab exercises and quality metric scripts.	≥ 3.11
Java JDK 17+	Language Runtime	Required for JUnit-based exercises (alternative to Python track).	<i>LTS 17 / 21</i>
pytest	Testing Framework	Python test runner — write, discover, and execute test cases.	$\geq 7.x$
JUnit 5	Testing Framework	Java test runner — annotations-based unit testing framework.	<i>5.x</i>
Coverage.py	Coverage Tool	Measures statement and branch coverage of Python test suites.	<i>Latest</i>
GitHub Account	Repository Hosting	All lab submissions will be pushed to a personal GitHub repository.	<i>Free tier</i>

Step 1 & 2 — VS Code & Git Installation

Follow along: installing your core tools



1. Go to code.visualstudio.com
2. Download installer for your OS (Win / Mac / Linux)
3. Run installer with default settings
4. Open VS Code → Extensions (Ctrl+Shift+X)
5. Install: Python | Extension Pack for Java
6. Install: GitLens | Pylance | Test Runner
7. Verify: View → Terminal opens a shell



1. Go to git-scm.com/downloads
2. Download for your OS & run installer
3. Open terminal: `git --version` (confirm ≥ 2.40)
4. Set identity:
5. `git config --global user.name "Your Name"`
6. `git config --global user.email "you@email.com"`
7. Verify: `git config --list`

Step 3 — Language Runtime Setup

Choose your track: Python or Java

Python Track

Install Python:

```
python.org/downloads → Python 3.11+
```

Verify:

```
python --version or python3 --version
```

Upgrade pip:

```
python -m pip install --upgrade pip
```

Install pytest:

```
pip install pytest pytest-cov
```

Verify pytest:

```
pytest --version
```

Install coverage:

```
pip install coverage
```

VS Code:

```
Set interpreter: Ctrl+Shift+P → Python: Select
```

Java Track

Install JDK:

```
adoptium.net → JDK 17 or 21 LTS
```

Verify:

```
java --version & javac --version
```

Install Maven:

```
maven.apache.org → mvn --version
```

Create project:

```
mvn archetype:generate (quickstart)
```

Add JUnit 5:

```
Add dependency in pom.xml
```

Run tests:

```
mvn test
```

VS Code:

```
Install: Extension Pack for Java
```

Step 4 — GitHub Repository Setup

Version control your lab work from day one

1

Create GitHub Account

Go to github.com → Sign Up → use your university email address.

2

Create Repository

New repo → Name: `sqe-labs-[yourname]` → Public → Add README → Create.

3

Clone Locally

```
git clone https://github.com/[username]/sqe-labs-[yourname].git
```

4

Set Up Folder Structure

```
mkdir week1 week2 ... week16 then touch week1/README.md
```

5

First Commit & Push

```
git add . → git commit -m "Week 1: initial setup" → git push origin main
```

6

Share Link with Instructor

Copy your repository URL — paste it into the lab submission form on the LMS.

Step 5 — Write & Run Your First Test

Verify the testing framework is working

Python — pytest

```
# week1/calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# week1/test_calculator.py
from calculator import add, subtract

def test_add_positive():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(10, 4) == 6

# Run from terminal:
# cd week1
# pytest test_calculator.py -v
```

Java — JUnit 5

```
// Calculator.java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public int subtract(int a, int b){
        return a - b;
    }
}

// CalculatorTest.java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.
    Assertions.*;
class CalculatorTest {
    @Test void testAdd() {
        assertEquals(5,
            new Calculator().add(2,3));
    }
}
```

Interpreting Test Output

Understanding what pytest and JUnit tell you

✓ PASSED Output (pytest -v)

```
==== test session starts ====  
collected 2 items  
  
test_calculator.py::test_add_positive PASSED  
test_calculator.py::test_subtract PASSED  
  
===== 2 passed in 0.12s =====
```

✗ FAILED Output (pytest -v)

```
==== test session starts ====  
FAILED test_calculator.py::test_add  
  AssertionError: assert 6 == 5  
    + where 6 = add(2, 3)  
  
===== 1 failed, 1 passed =====
```

How to Read the Output

PASSED	Test ran — actual output matched expected output. ✓
FAILED	Test ran — actual output did NOT match expected output. ✗
ERROR	Test could not run — syntax error, import error, or exception. ⚠
N passed in Xs	Summary line — total passed, total time taken.

VS Code — Recommended Extensions

Power-up your editor for SQE work

Python

by Microsoft

IntelliSense, linting, debugging, Jupyter support.

Python Test Explorer

by Little Fox Team

Run and debug pytest tests directly in the VS Code sidebar.

GitLens

by GitKraken

Inline blame, history, and comparison — supercharges Git.

REST Client

by Huachao Mao

Send HTTP requests — useful for API testing later in course.

Pylance

by Microsoft

Fast Python language server — better autocomplete & type hints.

Extension Pack for Java

by Microsoft

Bundles JDK tools, Maven, Debugger, Test Runner for Java.

Coverage Gutters

by ryanluker

Highlights covered/uncovered lines directly in the editor.

Thunder Client

by Thunder Client

Lightweight REST API testing within VS Code.

Tool Verification Checklist

Complete this before leaving the lab — submit as your deliverable

Run each command below in your terminal and record the output in your checklist worksheet.

Check Item	Command	Expected Output	Status
VS Code version	<code>code --version</code>	<i>1.xx.x or higher</i>	☐ ✓ / ☐ ✗
Git version	<code>git --version</code>	<i>git version 2.40+</i>	☐ ✓ / ☐ ✗
Git user name set	<code>git config user.name</code>	<i>Your Full Name</i>	☐ ✓ / ☐ ✗
Git user email set	<code>git config user.email</code>	<i>your@email.com</i>	☐ ✓ / ☐ ✗
Python version	<code>python --version</code>	<i>Python 3.11+</i>	☐ ✓ / ☐ ✗
pip version	<code>pip --version</code>	<i>pip 23+</i>	☐ ✓ / ☐ ✗
pytest installed	<code>pytest --version</code>	<i>pytest 7.x</i>	☐ ✓ / ☐ ✗
Coverage installed	<code>coverage --version</code>	<i>Coverage.py 7.x</i>	☐ ✓ / ☐ ✗
Test file runs	<code>pytest test_calculator.py -v</code>	<i>2 passed in 0.xxs</i>	☐ ✓ / ☐ ✗
GitHub repo accessible	<code>git remote -v</code>	<i>origin https://github.com/...</i>	☐ ✓ / ☐ ✗

Common Issues & Troubleshooting

Most frequent setup problems and their fixes



'python' not recognised (Windows)

Fix: Use python3 instead, or add Python to PATH: reinstall Python → check 'Add to PATH'.



'pytest' not found after pip install

Fix: Run python -m pytest instead. Or install in correct environment: pip3 install pytest.



Git push asks for username/password repeatedly

Fix: Set up a Personal Access Token (PAT) on GitHub → Settings → Developer Settings → Tokens.



VS Code does not detect Python interpreter

Fix: Ctrl+Shift+P → Python: Select Interpreter → choose the correct python3 path.



Java: 'javac' not found

Fix: Ensure JDK (not just JRE) is installed. Set JAVA_HOME environment variable to JDK folder.



Tests fail with ModuleNotFoundError

Fix: Run pytest from the project root, not from within the week1 folder. Add __init__.py if needed.

Lab Wrap-up



Submission: Push your repository to GitHub and paste the URL into the LMS before leaving.

Today You Achieved (CLO-1 P-2)

- Installed VS Code, Git, Python/Java
- Set up GitHub repository
- Configured pytest or JUnit
- Wrote and ran first test case

Coming Up — Week 2 Lab

- Quality Attribute Analysis Lab
- Evaluate software against ISO 25010 criteria
- Use quality checklists on a real app
- Submit a quality attribute assessment report

Pre-lab for Week 2: Read ISO 25010 Sections 1–4 · Identify 3 apps you use daily